

1 Lab Problem: Implement a Car Class with Constructors

Create a class Car with the following specifications:

1. **Private attributes:**
 - brand (string)
 - model (string)
 - price (double)
2. **Constructors:**
 - **Default Constructor:** Initializes brand = "Unknown", model = "Unknown", and price = 0.0.
 - **Parameterized Constructor:** Takes values for all attributes as arguments.
3. **Methods:**
 - **display():** Prints car details in the format:
Brand: [brand], Model: [model], Price: \$[price]
4. **In main(), do the following:**
 - Create one object using the default constructor.
 - Create another object using the parameterized constructor.
 - Display both objects.

2 Lab Problem: Create a Rectangle Class with Constructors

Design a class Rectangle with the following features:

1. **Private attributes:**
 - length (double)
 - width (double)
2. **Constructors:**
 - **Default Constructor:** Sets length = 1.0 and width = 1.0.
 - **Parameterized Constructor:** Takes length and width as input.
3. **Methods:**
 - **getArea():** Returns the area of the rectangle (length * width).
 - **display():** Prints rectangle dimensions and area.
4. **In main(), do the following:**
 - Create one Rectangle object using the default constructor.
 - Create another Rectangle object using the parameterized constructor.
 - Display their details.

3 Lab Problem: Implement a Student Class with Constructor

Develop a Student class with the following:

1. **Private attributes:**
 - id (int)

- name (string)
- marks (double)
- 2. **Constructors:**
 - **Default Constructor:** Sets id = 0, name = "Unknown", and marks = 0.0.
 - **Parameterized Constructor:** Takes values for all attributes as input.
- 3. **Methods:**
 - **display():** Prints student details in the format:
 - ID: [id], Name: [name], Marks: [marks]
- 4. **In main(), do the following:**
 - Create one student using the default constructor.
 - Create another student using the parameterized constructor.
 - Display both students.

4 Lab Problem: Create a BankAccount Class with Constructors

Implement a BankAccount class with the following:

1. **Private attributes:**
 - **accountNumber** (int)
 - **balance** (double)
2. **Constructors:**
 - **Default Constructor:** Sets accountNumber = 0 and balance = 0.0.
 - **Parameterized Constructor:** Takes account number and initial balance.
3. **Methods:**
 - **display():** Prints account details in the format:
 - **Account Number:** [accountNumber], **Balance:** \$[balance]
4. **In main(), do the following:**
 - Create one bank account using the default constructor.
 - Create another bank account using the parameterized constructor.
 - Display both accounts.

5 Lab Problem: Implement a Laptop Class with Constructors

Create a Laptop class with the following features:

1. **Private attributes:**
 - **brand** (string)
 - **processor** (string)
 - **ram** (int)
2. **Constructors:**
 - **Default Constructor:** Sets brand = "Unknown", processor = "Unknown", ram = 0.
 - **Parameterized Constructor:** Takes values for all attributes as input.
3. **Methods:**
 - **display():** Prints laptop details in the format:
 - **Brand:** [brand], **Processor:** [processor], **RAM:** [ram]GB

4. In main(), do the following:

- Create one laptop using the default constructor.
- Create another laptop using the parameterized constructor.
- Display both laptops.

- **Copy Constructor**

1 Problem: Implement a Car Class with Constructors

Create a class Car with the following:

- Private attributes: brand (string), model (string), price (double).
- Constructors:
 - Default Constructor (Initialize with "Unknown", "Unknown", 0.0).
 - Parameterized Constructor (Takes values as arguments).
 - Copy Constructor (Creates a copy of an existing object).
- Method: display(), which prints car details.
- main() should:
 - Create an object using the parameterized constructor.
 - Create another object using the copy constructor.
 - Display both objects.

2 Problem: Create a Rectangle Class with Copy Constructor

Task:

Design a class Rectangle with:

- Private attributes: length (double), width (double).
- Constructors:
 - Default Constructor (Set length and width to 1.0).
 - Parameterized Constructor (Takes values as input).
 - Copy Constructor (Copies values from another Rectangle object).
- Methods:
 - getArea(): Returns the area (length * width).
 - display(): Displays the rectangle's dimensions and area.
- main() should:
 - Create a rectangle object with the parameterized constructor.
 - Create another rectangle using the copy constructor.
 - Display their details.

3 Problem: Student Database with Copy Constructor

Task:

Develop a Student class with:

- Private attributes: id (int), name (string), marks (double).
- Constructors:
 - Default Constructor (Initialize id=0, name="Unknown", marks=0.0).
 - Parameterized Constructor (Takes input for all attributes).

- Copy Constructor (Copies data from another student).
- **Methods:**
 - `display()`: Prints student details.
- **main() should:**
 - Create two students using the parameterized constructor.
 - Create a third student by copying the first one.
 - Display details of all students.

4 Problem: Bank Account System Using Copy Constructor

Task:

Create a `BankAccount` class with:

- **Private attributes:** `accountNumber` (int), `balance` (double).
- **Constructors:**
 - Default Constructor (Initialize `accountNumber` = 0, `balance` = 0.0).
 - Parameterized Constructor (Takes account number and initial balance).
 - Copy Constructor (Copies an existing account).
- **Methods:**
 - `deposit(double amount)`: Adds amount to balance.
 - `withdraw(double amount)`: Deducts amount from balance (only if enough funds exist).
 - `display()`: Prints account details.
- **main() should:**
 - Create an account using the parameterized constructor.
 - Deposit and withdraw money.
 - Create another account using the copy constructor.
 - Display both accounts.

5 Problem: Laptop Class with Dynamic Memory (Deep Copy)

Task:

Implement a `Laptop` class with:

- **Private attributes:** `brand` (string), `processor` (string), `ram` (int*).
- **Constructors:**
 - Default Constructor (Set default values).
 - Parameterized Constructor (Dynamically allocate ram).
 - Copy Constructor (Deep Copy) to prevent memory issues.
- **Methods:**
 - `display()`: Prints laptop details.
- **main() should:**
 - Create a laptop using the parameterized constructor.
 - Create another laptop using the copy constructor.
 - Display both laptops.

Problem Statement:

Create a **Student** class with the following specifications:

1 Private Data Members:

- id (int)
- name (string)
- cgpa (double)

2 Constructors:

- **Default Constructor:** Initializes values to 0, "Unknown", and 0.0.
- **Parameterized Constructor:** Assigns values provided by the user.
- **Copy Constructor:** Copies data from one object to another.

3 Member Functions:

```
less
```

```
Student ID: [id]
```

```
Name: [name]
```

```
CGPA: [cgpa]
```

4 In main():

- Create an object **s1** using the **parameterized constructor**.
- Create another object **s2** using the **copy constructor**.
- Display the details of both objects.

C++ Implementation

```
#include <iostream>
using namespace std;

class Student {
private:
    int id;
    string name;
    double cgpa;

public:
    // Default Constructor
    Student() {
        id = 0;
        name = "Unknown";
        cgpa = 0.0;
    }

    // Parameterized Constructor
    Student(int i, string n, double c) {
        id = i;
        name = n;
        cgpa = c;
    }
}
```

```

// Copy Constructor
Student(const Student &s) {
    id = s.id;
    name = s.name;
    cgpa = s.cgpa;
}

// Display function
void display() {
    cout << "Student ID: " << id << endl;
    cout << "Name: " << name << endl;
    cout << "CGPA: " << cgpa << endl;
    cout << "-----" << endl;
}
};

// Main function to test constructors
int main() {
    // Creating object using parameterized constructor
    Student s1(101, "Alice", 3.85);
    cout << "Details of Student 1:" << endl;
    s1.display();

    // Creating object using copy constructor

```

```

    // Creating object using copy constructor
    Student s2(s1);
    cout << "Details of Student 2 (Copied from Student 1):" << endl;
    s2.display();

    return 0;
}

```

Expected Output

yaml

Details of Student 1:

Student ID: 101

Name: Alice

CGPA: 3.85

Details of Student 2 (Copied from Student 1):

Student ID: 101

Name: Alice

CGPA: 3.85
